COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	7/16/2026	**DATE SUBMITTED:**	7/16/2026
NAME:	Formilleza, Huone Anthony	**INSTRUCTOR:**	Eng. Jimlord Quejado

ACTIVITY NO. _2_ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Screenshot

main.cpp test.cpp

main.cpp > main()

```cpp
#include <iostream>
#include <string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }; <- #11-15 Student(std::string newName = "John Doe", int new

    // Destructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

```

```cpp
    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    } <- #23-27 Student(const Student &copyStudent)

    // Display Attributes
    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
}; <- #4-33 class Student

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;
return 0;
} <- #35-41 int main()
```

Observation

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

The output shows exactly when each **constructor** and **destructor** runs. The **constructor** works first for **student1** and then the **copy constructor** runs when we make **student2** as a copy of **student1** and then creating **student3** also calls the **constructor**. When the program ends, all of the three **destructors** run, in reverse order of how we created the objects.

Table 2-1. Initial Driver Program

Screenshot

```cpp
int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};
    return 0;
} <- #35-41 int main()
```

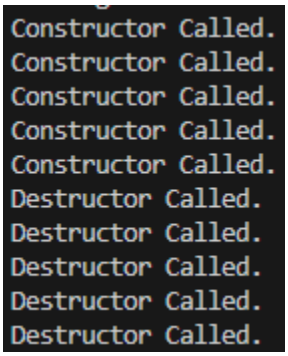
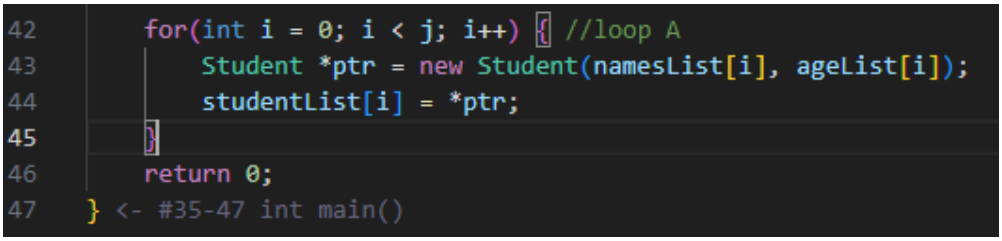
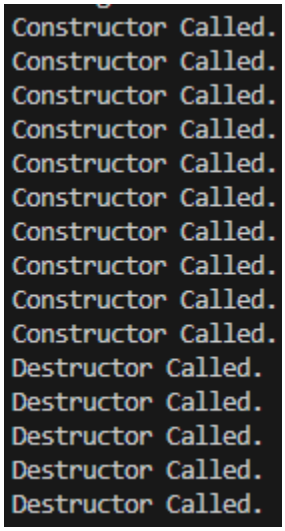
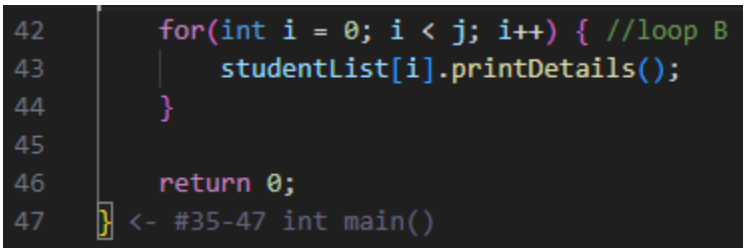
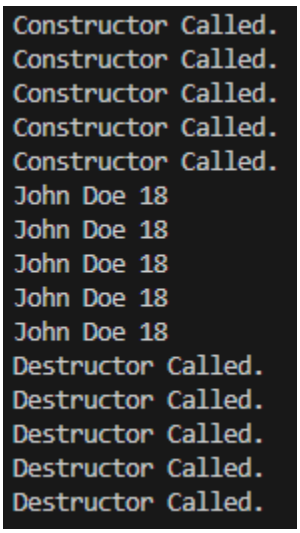
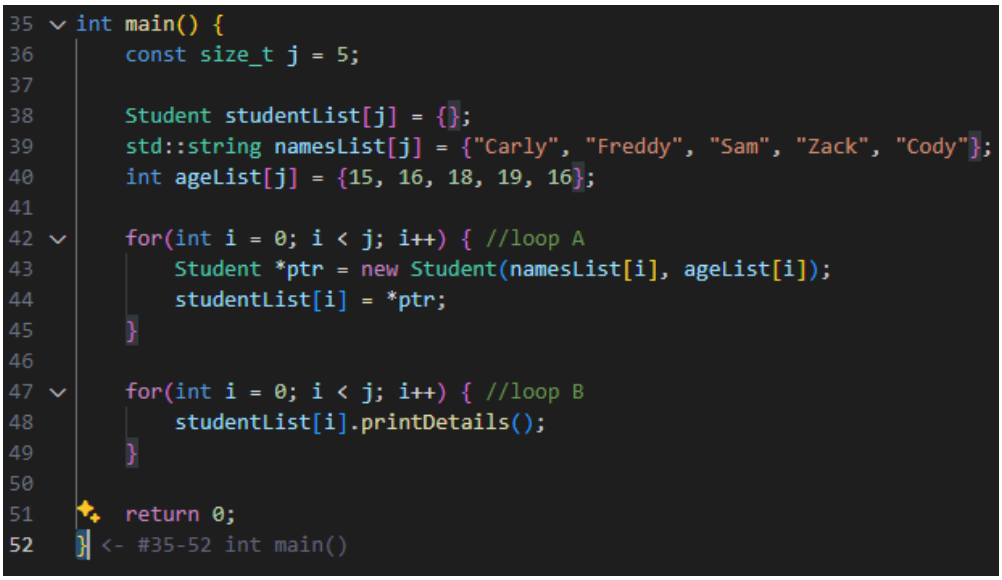
Observation	 In this one we made an array of 5 Student objects here, thats why we see **Constructor Called** 5 times each object in the array runs the **constructor** when it's created. When the program finishes, all of the 5 objects are deleted one by one, and then also the **Destructor Called** also appears 5 times.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	 In this one we used a loop with **new** to create each student object one by one. This makes the constructor run every time the loop repeats, this is why we see so many **Constructor**

	Called messages, but as we see here there's only 5 destructor calls at the end the extra objects made with new aren't being cleaned up here. This presents that using **new** always triggers the constructor, and these objects won't get destroyed automatically on their own.
Loop B	
Observation	
Output	

```
42      for(int i = 0; i < j; i++) { //loop B
43          studentList[i].printDetails();
44      }
45
46      return 0;
47  } <- #35-47 int main()
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
John Doe 18
John Doe 18
John Doe 18
John Doe 18
John Doe 18
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

```
35  int main() {
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41
42      for(int i = 0; i < j; i++) { //loop A
43          Student *ptr = new Student(namesList[i], ageList[i]);
44          studentList[i] = *ptr;
45      }
46
47      for(int i = 0; i < j; i++) { //loop B
48          studentList[i].printDetails();
49      }
50
51      return 0;
52  } <- #35-52 int main()
```

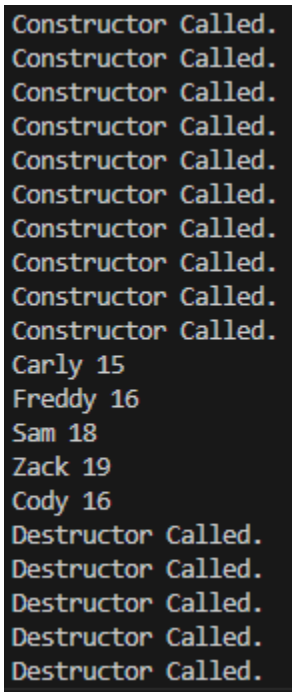
Observation	

Table 2-3. Final Driver Program

Modifications	
Observation	

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.